

Flutter Project Analysis & Review

Comprehensive analysis of the sih_project Flutter repository. Identifying critical issues, architectural gaps, and providing a roadmap to transform the prototype into a production-ready educational platform.

Repository: github.com/pilot32/sih_project

Framework: Flutter (Dart SDK ^3.8.1)

Backend: Firebase (edutivflutter)

Platform: Android, iOS, Web, Linux, macOS, Windows

Table of Contents

1. Executive Summary	3
2. Project Overview	3
2.1 Technology Stack	3
2.2 Project Structure	4
3. Critical Issues	4
3.1 Duplicate Class Definition (Build Risk)	5
3.2 Three Empty Placeholder Files	5
3.3 No Authentication Implementation	5
3.4 Stale Test Suite	5
3.5 Missing Internet Permission (Android)	5
4. Significant Problems	6
4.1 State Management Not Implemented	6
4.2 No Data Persistence Layer	6
4.3 Teacher Dashboard Completely Absent	6
4.4 Deprecated Material 2 ColorScheme Properties	7
4.5 Firebase Configuration Incomplete	7
4.6 Unused Assets and Dead Code	7

5. Missing Features	8
6. Security Concerns	8
6.1 Exposed Firebase API Keys	8
6.2 No Input Sanitization	9
6.3 Missing Network Security Configuration	9
7. Recommended Changes	9
7.1 Tier 1: Immediate Fixes (Week 1)	9
7.2 Tier 2: Architecture Improvements (Weeks 2-3)	10
7.3 Tier 3: Feature Development (Weeks 4-8)	10
8. Proposed Architecture	11
9. Suggested Dependency Additions	12
10. Conclusion	13

1. Executive Summary

This report presents a comprehensive analysis of the Flutter-based educational application repository located at github.com/pilot32/sih_project. The project, branded as "Nabha Digital Education" (also referred to as "Nabha Digital Learning Platform"), is intended to be a learning management system targeting students and teachers in India. It covers subjects such as Hindi, English, Mathematics, Science, Social Science, Commerce, and Computer Science.

The analysis reveals that the project is in an **early prototype stage**. While the UI layer for the student login screen and dashboard has been constructed with reasonable visual design, the application lacks critical backend infrastructure, authentication, data persistence, and most feature implementations. Firebase is initialized at the core level but no services (Authentication, Firestore, Storage, Analytics) are actually used. The state management library (flutter_riverpod) is declared as a dependency but never invoked anywhere in the codebase. All data presented in the dashboard is hardcoded mock data with no connection to any real data source.

The codebase exhibits several architectural inconsistencies including misplaced widget files in the data layer, duplicate class definitions, multiple empty placeholder files, an outdated test suite, and unused dependencies. The teacher role has a login form but navigates to the same student dashboard, indicating that the teacher experience was never developed. This document provides a detailed breakdown of every issue found, categorized by severity, along with actionable recommendations to transform this prototype into a production-ready application.

Total Dart Files	Empty Files	Critical Issues	Features Missing
18	3	5	11+

Table 1: Key Project Metrics at a Glance

2. Project Overview

2.1 Technology Stack

The project is built with Flutter (Dart SDK ^3.8.1) and targets Android, iOS, Web, Linux, macOS, and Windows platforms. It uses Firebase as the intended backend-as-a-service provider, with the Firebase project identified as "edutivflutter". The application uses Material Design 3 (useMaterial3 is enabled) for its UI framework. The following table summarizes the declared dependencies and their actual usage status within the codebase.

Dependency	Purpose	Status
------------	---------	--------

firebase_core ^4.3.0	Firebase initialization	Used (init only)
flutter_riverpod ^3.1.0	State management	Declared but NEVER used
percent_indicator ^4.2.3	Progress bar widget	Used once
cupertino_icons ^1.0.8	iOS style icons	Available, not used
firebase_auth	Authentication	NOT declared, NOT used
cloud_firestore	Database	NOT declared, NOT used
google_sign_in	Google OAuth	NOT declared, NOT used

Table 2: Dependency Analysis - Current vs. Required

2.2 Project Structure

The project follows a partial clean architecture pattern with directories for core (theme, routes, constants), data/models, presentation/pages, screens, and widgets. However, this architecture is inconsistently applied. Widget files exist in the data/models directory, screens and presentation/pages both serve as screen containers, and there is no repository layer, service layer, or use case layer. The overall file organization suggests multiple refactoring iterations without cleanup, leaving dead code and misplaced files throughout the project.

Directory	Purpose	Assessment
lib/core/	Theme, routes, constants	Well organized
lib/data/models/	Data classes	Misplaced widget file
lib/screens/	UI screens (auth, dashboard)	Contains old prototype
lib/presentation/pages/	Intended screen layer	Contains 1 empty file
lib/widgets/	Reusable components	3 empty files, unused cardsView/
lib/widgets/cardsView/	Generic card widgets	Unused legacy code

Table 3: Directory Structure Assessment

3. Critical Issues

This section covers the most severe problems that either prevent the application from building correctly or represent fundamental architectural failures that must be addressed before any feature development can proceed meaningfully. Each issue is explained with its root cause and recommended resolution.

3.1 Duplicate Class Definition (Build Risk)

The class **SubjectsSection** is defined in two separate files with different constructor signatures. The primary definition resides in **lib/widgets/subjects_section.dart** and accepts a parameter named "badgeText". A second definition exists in **lib/data/models/subjects_section.dart** (a widget file incorrectly placed in the data layer) and uses a parameter named "badge". While the current dashboard screen only imports the widgets version, this creates a latent compilation risk. If any future import references both files, the Dart compiler will fail with a duplicate class name error. The file in the data layer must be deleted since it is architecturally misplaced and functionally redundant.

3.2 Three Empty Placeholder Files

Three files in the project contain only a single empty line and no functional code whatsoever. The file **lib/widgets/quick_action_button.dart** is actually imported by the student dashboard screen, which means the application may reference a non-existent widget at runtime. The file **lib/presentation/pages/student_dashboard_page.dart** is completely empty, suggesting an abandoned architectural refactoring attempt where screens were supposed to move to the presentation layer but the migration was never completed. Similarly, **lib/widgets/ui/custom_avatar.dart** is empty. All three files must either be populated with their intended implementations or removed entirely to prevent import errors and reduce codebase confusion.

3.3 No Authentication Implementation

The login screen in **lib/screens/auth/login_page2.dart** presents a well-designed UI with Student and Teacher tab-based login forms. The Student tab collects a name and phone number, while the Teacher tab collects a Teacher ID and phone number. However, the **_handleLogin()** method does not perform any credential verification whatsoever. It simply displays a Snackbar message and navigates directly to the student dashboard. Firebase Auth is not declared in pubspec.yaml, and no API calls, token validation, or session management exists anywhere in the codebase. This means any user can bypass the login screen with arbitrary input, making the authentication layer purely cosmetic and providing zero security.

3.4 Stale Test Suite

The sole test file **test/widget_test.dart** is the default Flutter counter app test generated by the "flutter create" command. It tests for the presence of a counter text widget and a floating action button with a "+" icon, neither of which exist in the current application. Running "flutter test" will produce a failure, which means the project has no passing tests at all. For any serious development effort, this test file must be replaced with tests that cover the actual login flow, dashboard rendering, subject card interactions, and widget behaviors.

3.5 Missing Internet Permission (Android)

The `AndroidManifest.xml` file is missing the essential **INTERNET** permission declaration. The older dashboard prototype in `lib/screens/homescreen/student_dashboard.dart` loads images from Unsplash URLs using the `NetworkImage` widget. Without the internet permission, these network requests will fail silently on Android devices. Additionally, the manifest does not enforce HTTPS traffic or configure cleartext traffic policies, which is a security concern for a production application. The recommended fix is to add the **INTERNET** permission and set `android:usesCleartextTraffic="false"` in the application tag.

4. Significant Problems

Beyond the critical build-breaking issues, the project has several significant problems that affect code quality, maintainability, and the ability to develop features incrementally. While these issues may not prevent the app from launching in its current form, they create technical debt that will compound rapidly as the codebase grows.

4.1 State Management Not Implemented

The `flutter_riverpod` package (version `^3.1.0`) is declared in `pubspec.yaml`, indicating an intent to use a robust state management solution. However, a thorough search of every Dart file in the project reveals that Riverpod is never imported, never referenced, and never used. The entire application relies on local `setState()` calls within `StatefulWidget` instances and hardcoded data initialized in `initState()` methods. This approach does not scale beyond a single screen and makes it impossible to share state between screens, persist data across navigation events, or implement reactive updates from a backend. The team should either commit to implementing Riverpod properly (creating providers for auth state, subject data, user progress, etc.) or remove the dependency to avoid confusion.

4.2 No Data Persistence Layer

All data displayed in the student dashboard (subject cards, progress percentages, streak counts, lesson totals, completion counts) is hardcoded in the `_initializeData()` method of the `StudentDashboardScreen` class. There is no Firestore integration, no local database (SQLite/Hive), no shared preferences storage, and no API service layer. This means that every time the application restarts, all "progress" resets to the same hardcoded values. For a learning management system where tracking student progress is a core requirement, this is a fundamental gap. The project needs a repository pattern with data sources for both remote (Firestore) and local (cached/offline) storage.

4.3 Teacher Dashboard Completely Absent

The login screen includes a Teacher tab with a form for Teacher ID and phone number, but the `_handleLogin()` method for the teacher role navigates to the exact same `StudentDashboardScreen`. There is no teacher-specific route defined in `routes.dart`, no teacher dashboard screen, and no teacher-specific

functionality anywhere in the codebase. For a Smart India Hackathon project that likely requires both student and teacher interfaces, this is a major missing feature. A teacher dashboard would typically need class management, student progress overview, assignment creation, content management, and analytics capabilities.

4.4 Deprecated Material 2 ColorScheme Properties

The login screen (`login_page2.dart`) uses `colorScheme.surfaceVariant` and `colorScheme.onSurfaceVariant`, which are properties from the Material 2 ColorScheme API. Since the project uses `useMaterial3: true` in `pubspec.yaml`, these properties are deprecated and may cause runtime warnings or errors on newer Flutter versions (3.22+). In Material 3, these have been replaced with `surfaceContainerHighest` and `onSurfaceVariant` respectively. The login screen should be updated to use the correct Material 3 color properties to ensure compatibility with the latest Flutter stable releases.

4.5 Firebase Configuration Incomplete

While `firebase_core` is initialized in `main.dart` and a `firebase_options.dart` file exists with platform-specific configurations, the Firebase integration goes no further. The `google-services.json` for Android has an empty `oauth_client` array, meaning no sign-in providers are configured. No Firebase services beyond core initialization are used: no Firebase Auth for user authentication, no Cloud Firestore for data storage, no Firebase Storage for media uploads, no Firebase Analytics for usage tracking, and no Firebase Cloud Messaging for push notifications. The `firebase_options.dart` file also contains hardcoded API keys for Web, Android, iOS/macOS, and Windows platforms, which are committed to the public repository and pose a security risk if the Firebase project contains sensitive data.

4.6 Unused Assets and Dead Code

Several asset files in the `assets/images/` directory are declared but never referenced in the active codebase. Specifically, `hindi_bg.jpg`, `english_bg.jpg`, `math_bg.jpg`, and `stats_banner_bg.jpg` are available but unused. The `SubjectModel` class defines a `backgroundImage` property designed to hold these paths, but the active dashboard screen does not populate this field for any subject. Additionally, the `lib/widgets/cardsView/` directory contains three generic card widgets (`courseCard.dart`, `subjectCard.dart`, `categoryCard.dart`) that are never imported or used anywhere in the application. The `percent_indicator` package is used only once while a custom progress widget was also built, creating redundant dependency weight. These unused assets and code should be removed to reduce the application binary size and improve codebase clarity.

5. Missing Features

A learning management platform requires a comprehensive set of features to be functional. This section maps the expected features against the current implementation status, highlighting what needs to be built to make the application viable for real-world use.

Feature	Status	Details
Student Login UI	Implemented	Name + phone form with basic validation
Teacher Login UI	Implemented	Teacher ID + phone form
Real Authentication	Missing	No Firebase Auth, no credential verification
Student Dashboard	Implemented	Stats, subjects grid, quick actions with mock data
Teacher Dashboard	Missing	No route, no screen, no functionality
Subject Detail View	Missing	Tapping Continue does nothing, no lesson pages
Progress Tracking	Mock Only	Hardcoded values, not persisted
Streak / Rewards System	Mock Only	No rewards page, hardcoded streak of 15
Settings Screen	Placeholder	Shows only static text, no settings controls
Dark Theme	Defined Unused	darkTheme exists in app_theme.dart but not wired
Offline Mode	Missing	Footer claims availability but nothing implemented
Data Persistence	Missing	No Firestore, no local DB, no shared preferences
Push Notifications	Missing	No FCM integration
Profile Management	Missing	No user profile screen or edit capability

Table 4: Feature Implementation Status Matrix

6. Security Concerns

The current codebase has several security vulnerabilities that must be addressed before the application can be considered safe for any deployment, even for testing purposes. These issues range from exposed API keys to missing input validation and absent network security configurations.

6.1 Exposed Firebase API Keys

The `firebase_options.dart` file contains hardcoded API keys for all supported platforms (Web, Android, iOS/macOS, Windows). These keys are committed to the public GitHub repository and can be accessed by

anyone. While Firebase API keys are somewhat different from secret keys (they identify the project, not authenticate users), exposing them allows potential abuse including unauthorized quota consumption, database access if security rules are misconfigured, and service enumeration. The recommended approach is to use Firebase App Check, configure proper security rules for Firestore and Storage, and ensure that the API key restrictions in the Google Cloud Console limit usage to your specific application package identifiers.

6.2 No Input Sanitization

The login forms perform only basic length checks on user input. There is no sanitization against SQL injection (relevant if a backend is later added), no XSS prevention for any displayed text, and no validation of phone number format beyond checking string length. The Teacher ID field accepts any arbitrary string without format validation. When connecting to Firebase, input should be validated and sanitized before being stored or transmitted. Phone numbers should be validated against Indian mobile number formats (10 digits starting with 6-9), and names should be checked for reasonable character sets.

6.3 Missing Network Security Configuration

The `AndroidManifest.xml` does not enforce HTTPS-only traffic by setting `android:usesCleartextTraffic="false"`. While modern Android versions (9+) default to blocking cleartext traffic, explicitly configuring this setting ensures that all API calls and data transfers use encrypted connections. Additionally, no certificate pinning is implemented, which is recommended for applications handling educational data that may include personally identifiable information about students.

7. Recommended Changes

Based on the comprehensive analysis above, the following changes are organized into three priority tiers. Immediate fixes address issues that prevent the app from building or functioning correctly. Architecture improvements establish the foundation for scalable feature development. Feature development items are the new capabilities needed to make the application complete and usable.

7.1 Tier 1: Immediate Fixes (Week 1)

Fix	Action	Priority
Delete duplicate SubjectsSection	Remove lib/data/models/subjects_section.dart	High
Remove empty files	Delete quick_action_button.dart, student_dashboard_page.dart, custom_avatar.dart	High
Fix stale test	Replace test/widget_test.dart with actual app tests	High

Add INTERNET permission	Add to AndroidManifest.xml	High
Fix deprecated ColorScheme	Update login_page2.dart to Material 3 properties	Medium
Remove old prototype	Delete lib/screens/homescreen/student_dashboard.dart	Medium
Clean unused cardsView/	Delete lib/widgets/cardsView/ directory	Low
Remove unused assets	Delete or use hindi_bg.jpg, english_bg.jpg, math_bg.jpg, stats_banner_bg.jpg	Low

Table 5: Tier 1 - Immediate Fix Actions

7.2 Tier 2: Architecture Improvements (Weeks 2-3)

Task	Description	Priority
Implement Riverpod state management	Create providers for AuthState, SubjectList, UserProgress, AppSettings	Critical
Create repository layer	Build AuthRepository, SubjectRepository, ProgressRepository with abstract interfaces	Critical
Integrate Firebase Auth	Add firebase_auth dependency, implement phone OTP and Google Sign-In flows	Critical
Integrate Cloud Firestore	Add cloud_firestore, design data models for users, subjects, progress, lessons	Critical
Implement proper navigation	Use GoRouter or auto_route for type-safe deep linking and nested navigation	High
Create service layer	Build ApiService for backend communication, NotificationService for FCM	High
Configure Firebase properly	Set up Auth providers in console, write Firestore security rules, enable App Check	High
Remove percent_indicator	Replace sole usage with existing CustomProgress widget	Low

Table 6: Tier 2 - Architecture Improvement Tasks

7.3 Tier 3: Feature Development (Weeks 4-8)

Feature	Description
Subject Detail / Lesson View	Build screen with lesson list, video player, quiz, and mark-as-complete functionality
Teacher Dashboard	Create class overview, student progress, assignment creation, and content management screens

Real Progress Tracking	Connect progress bars and streak counters to Firestore with real-time updates
Rewards / Stars System	Build gamification layer with stars, badges, leaderboards, and unlockable content
Settings Screen	Implement theme toggle, notification preferences, language selection, account management
Offline Mode	Use Hive or Isar for local caching of lessons and progress data with sync on reconnect
User Profile Management	Build profile screen with avatar upload, personal details, and learning statistics
Push Notifications	Integrate FCM for lesson reminders, streak alerts, and assignment deadlines
Search and Filter	Add search functionality across subjects, lessons, and study materials
Analytics Dashboard	Build student performance analytics with charts and study pattern insights

Table 7: Tier 3 - Feature Development Roadmap

8. Proposed Architecture

To address the architectural issues identified in this analysis, the following clean architecture structure is recommended. This structure separates concerns into distinct layers, making the codebase testable, maintainable, and scalable for future feature additions.

Path	Contents
lib/	Root package directory
core/	Constants, themes, routes, utilities, validators
data/	Repositories, data sources (remote + local), models
data/datasources/	FirebaseDataSource, LocalCacheDataSource
data/repositories/	AuthRepositoryImpl, SubjectRepositoryImpl
domain/	Entity classes, repository interfaces, use cases
domain/entities/	User, Subject, Lesson, Progress
domain/repositories/	Abstract AuthRepository, SubjectRepository
domain/usecases/	LoginUseCase, GetSubjectsUseCase, UpdateProgressUseCase
presentation/	Screens, widgets, providers/controllers
presentation/providers/	AuthProvider, SubjectProvider, ProgressProvider (Riverpod)

presentation/screens/	Auth screens, Dashboard, SubjectDetail, Settings, Profile
presentation/widgets/	Reusable UI components

Table 8: Recommended Clean Architecture Directory Structure

This architecture follows the dependency inversion principle: the domain layer defines interfaces that the data layer implements. The presentation layer depends on the domain layer (not the data layer), making it easy to swap data sources without modifying UI code. Riverpod providers sit at the presentation layer and mediate between the UI and the domain use cases, providing reactive state updates whenever underlying data changes. This pattern has been proven effective in production Flutter applications at scale.

9. Suggested Dependency Additions

To support the recommended architecture and feature set, the following dependencies should be added to pubspec.yaml. Each dependency is chosen based on its maturity, community support, and compatibility with the existing Flutter 3.8+ / Dart 3.x environment.

Package	Purpose
firebase_auth	User authentication with phone OTP and Google Sign-In
cloud_firestore	Cloud database for users, subjects, progress, lessons
firebase_analytics	Usage tracking and event logging
firebase_messaging	Push notifications for reminders and updates
google_sign_in	Google OAuth authentication provider
go_router	Declarative routing with deep link support
flutter_secure_storage	Encrypted local storage for auth tokens
hive / isar	Local database for offline data caching
connectivity_plus	Network connectivity detection for offline mode
cached_network_image	Efficient image loading and caching for subject cards
flutter_svg	SVG rendering for icons and illustrations
freezed / json_serializable	Immutable data classes with JSON serialization
dio	HTTP client for any REST API integrations

Table 9: Recommended Dependencies to Add

10. Conclusion

The `sih_project` repository represents an early-stage educational application with a functional student-facing UI prototype but significant gaps in backend integration, state management, data persistence, and feature completeness. The login screen and student dashboard demonstrate competent Flutter UI development skills and thoughtful visual design with gradient themes, responsive layouts, and branded assets. However, without authentication, data persistence, or most core features, the application cannot serve its intended purpose as a learning management platform.

The most critical finding is that the project has zero functional backend connectivity. Firebase is initialized but unused, state management is declared but not implemented, and all data is hardcoded. This means the app presents a polished facade over empty functionality. The recommended approach is to follow the three-tier roadmap outlined in Section 7: first fix all build-breaking issues and clean up dead code (Tier 1, 1 week), then establish the proper architectural foundation with Riverpod, repositories, and Firebase services (Tier 2, 2-3 weeks), and finally build out the complete feature set including subject lessons, teacher dashboard, gamification, and offline support (Tier 3, 4-8 weeks).

With disciplined execution of this roadmap, the project can evolve from its current prototype state into a production-ready educational platform suitable for the Smart India Hackathon evaluation and, ultimately, real student and teacher use.